

flatmem: A Constant-RAM Content-Addressable Memory Substrate for Digital Organisms and Artificial Life Agents

Prince Siddhpara

Mura ALife Labs

May 2026

Abstract

We present **flatmem**, a constant-RAM content-addressable memory substrate for online-learning digital organisms. The architecture integrates five mathematical primitives—Sparse Distributed Memory (SDM) [20], Vector Symbolic Architecture (VSA) role-binding [34, 21], computed-identity keys, read-time mean-removal [31], and traffic-class bank separation—into a single substrate whose total bytes are *fixed* regardless of how much information is written. A trained instance occupies 192 MB of substrate carrying co-occurrence statistics, IS-A taxonomy, sensory grounding, verb-rotor arithmetic, transition probabilities, and classification labels for a natural-language plus vision organism; full inference RAM is 233 MB. We empirically demonstrate constant-RAM behaviour up to 100K text exposures (vocabulary of 19,411 words), substrate-flatness through cross-modal training, modality-blind extension to a vision channel (90% test accuracy on 8×8 digit classification), and—most critically—catastrophic-forgetting-free continual learning: 5 original facts retained with 100% recall through 5,000+ distractor writes spanning four different channels. We argue **flatmem** is the universal flat-memory primitive for Artificial Life simulations, embedded agents, and any system that needs lifelong learning within a bounded RAM budget. Code is open-source under MIT license at <https://github.com/HitoshiFTW/Flatmem-Mura-ALife-Labs> and installable via `pip install flatmem`.

1 Introduction

Modern machine learning systems scale memory with data. Embedding tables in language models grow with vocabulary [30, 33]; replay buffers grow with corpus size; vector databases grow linearly with stored items. Biological brains do neither. An adult human cortex contains roughly 86×10^9 neurons and approximately 10^{14} synapses fixed from development [3]; a lifetime of experience is encoded in the *strengths* of synaptic connections within that fixed matrix, not in newly allocated storage. This gap has practical consequences for online-learning systems in robotics, multi-agent simulations, and edge deployment, where unbounded growth is unacceptable.

Existing approaches sit at extremes. Static word-embedding tables (WORD2VEC [30], GLoVE [33], contextual variants such as BERT [6]) require an entry per token or sub-word unit [40]—memory $\Theta(|V|)$ in vocabulary $|V|$. Vector databases inherit this growth. Transformer language models [43, 4] compress vocabulary into hidden layers but suffer *catastrophic forgetting* under sequential training [28, 10] without expensive remediation (Elastic Weight Consolidation [23], Memory Aware Synapses [2], gradient episodic memory [26], full retraining, or parameter-efficient fine-tuning with LoRA [16]). Sparse Distributed Memory (Kanerva 1988 [20]) provides

a biologically grounded fixed-capacity substrate but, in its classical binary form, supports only single-channel autoassociative recall.

This work proposes a different design point. The substrate is a fixed-size counter bank. Knowledge is *superposed* across many cells; each cell participates in many memories. Identity is *computed* from input strings or vectors rather than looked up in a table. Recall is *reconstructive*—lossy and gist-based, like human memory. The result is a substrate that:

- Learns online (single-write per fact, no gradients).
- Never forgets catastrophically (additive superposition).
- Has bounded RAM at any data volume.
- Runs on a laptop CPU (no GPU required).
- Hosts multiple channels in one substrate (text, taxonomy, sensory, arithmetic, transitions, classification).
- Is modality-blind (text and pixels coexist).

Contributions. The contributions of this work are: **(1)** the empirical discovery and structural resolution of *traffic-class bank separation* as a precondition for multi-channel VSA-SDM substrates (Section 3); **(2)** the dynamic adaptation of the static all-but-the-top common-direction removal of Arora et al. [31] to live recall within a content-addressable memory, yielding a $14\times$ improvement in related/unrelated pair separation (Section 5); **(3)** zero-storage computed-identity keys with a whole-word phasor weight parameter that eliminates lexical-overlap collisions (Section 3); **(4)** the systemic integration of these into a complete online-learning, multi-channel, modality-blind organism within a fixed 192 MB budget; and **(5)** empirical demonstration of catastrophic-forgetting-free continual learning across four distinct channels with 5,000+ distractor writes (Section 5.4, Figure 3).

Source code, the four ALife integration example scripts, and the figure generation scripts used in this paper are available at the project repository under MIT license.

2 Background

2.1 Sparse Distributed Memory and the cerebellar model

Kanerva (1988) [20] introduced SDM as a mathematical model of cerebellar function, building on Marr [27] and Albus [1], with biological grounding from Eccles, Ito, and Szentágothai [7, 18]. The substrate is a bank of M hard locations with random binary addresses; writing activates the subset of locations within Hamming radius r of the query; reading averages the contents of activated locations. Capacity is bounded but graceful: information degrades smoothly as the substrate fills [19]. Frady and Sommer [9] extended SDM to complex phasor space (Threshold Phasor Associative Memory, TPAM), gaining native support for continuous quantities and significantly higher capacity than binary SDM. Hopfield networks [15, 25, 38] form a parallel lineage of associative memory at higher capacity but different operational semantics (energy descent rather than location-set averaging).

2.2 Vector Symbolic Architectures

Plate’s Holographic Reduced Representations (HRR) [34], Kanerva’s Spatter Codes [21], Gayler’s framing [11], and Rachkovskij and Kussul’s binary thinning [35] established VSA as a family of distributed representations where structured data (trees, graphs, key-value bundles) can be

encoded as flat vectors via *binding* (typically circular convolution or component-wise multiplication) and *bundling* (superposition). Recent surveys by Schlegel et al. [39], Kleyko et al. [24], and Ge & Parhi [12] compare VSA variants and applications. Frady, Kent, Olshausen, and Sommer [8, 22] introduced Resonator Networks for efficient factorization of bound structures.

2.3 Common-direction removal in static embeddings

Mu, Bhat, and Viswanath [31] observed that pre-trained word embeddings share a dominant “common discourse” direction that suppresses semantic discrimination. Removing the top principal components (“all-but-the-top”) dramatically improves retrieval. The technique is presented as a static post-processing step for fixed embedding tables; this work adapts it to live recall in a content-addressable memory (Section 5).

2.4 Continual learning and catastrophic forgetting

McCloskey & Cohen [28] and French [10] characterised catastrophic forgetting in connectionist networks. Parisi et al. [32] survey contemporary continual-learning approaches. Common mitigations include weight-importance regularization [23, 2], gradient episodic replay [26], and parameter-efficient adaptation [16]—each adds machinery to the trained model. This work argues that the catastrophic-forgetting problem is an artefact of how neural networks encode memory (in shared weights modified by gradient descent); a substrate built on *additive* superposition over *sparse* hard locations bypasses the problem structurally.

2.5 Hyperdimensional Computing in Artificial Life and neuromorphic hardware

Yilmaz [44] applied VSA-based reservoir computing to cellular automata. Ultra-lightweight edge robotics increasingly leverages HDC for resource-constrained sensorimotor integration [37]. Specialised neuromorphic hardware including IBM TrueNorth [29] and Intel Loihi [5] provide platforms where flat memory primitives can run with substantial energy advantages over GPUs; Indiveri et al. [17] review the broader silicon-neuron landscape. The Hierarchical Temporal Memory tradition initiated by Hawkins and Ahmad [14] shares the high-sparsity distributed-representation philosophy.

2.6 Memory in deep learning systems

End-to-end memory networks [42], differentiable neural computers [13], and compressive transformers [36] address memory in deep architectures. All require gradient training and substantial parameter overhead, in contrast to the gradient-free, content-addressable design pursued here.

3 Architecture

3.1 Notation

We work in complex-valued phasor space. A hypervector (HV) is a vector in $\mathbb{C}^d$ where each component has unit magnitude ($|h_j| = 1$). The default dimension is $d = 512$. Similarity is the normalized real-part inner product: $\cos(a, b) = \text{Re}\langle a, b \rangle / d$. The element-wise complex multiplication $a \odot b$ is the VSA *bind* operation; componentwise unit-phasor renormalization is denoted $\text{renorm}(\cdot)$.

Algorithm 1 SDM write and read

```

1: function WRITE(addr, data)
2:   sims  $\leftarrow$  Re( $H^* \cdot$  addr)
3:   idx  $\leftarrow$  argpartitiontop- $k$ (sims)
4:    $C[\text{idx}] +=$  data
5: end function
6: function READ(addr)
7:   sims  $\leftarrow$  Re( $H^* \cdot$  addr)
8:   idx  $\leftarrow$  argpartitiontop- $k$ (sims)
9:   return renorm ( $\sum_{m \in \text{idx}} C[m]$ )
10: end function

```

3.2 Computed identity

Item identity is not stored. For a string w , the HV is computed as

$$\text{key}(w) = \text{renorm} \left(\sum_{tg \in \text{trigrams}(\#w\#)} P(tg) + \alpha \cdot P(\text{WORD}:w) \right), \quad (1)$$

where $P(\cdot)$ is a seeded random phasor projection $P(\text{tag}) = \exp(i \cdot \text{uniform}(-\pi, \pi)_d)$ with seed = SHA-256(tag). Character trigrams give morphological similarity for out-of-vocabulary (OOV) inputs; the whole-word phasor weight $\alpha = 4$ ensures distinct strings have near-orthogonal keys regardless of character overlap. Empirically, $\cos(\text{key}(\text{little}), \text{key}(\text{little}))$ drops from 0.85 (trigrams alone, $\alpha = 0$) to 0.09 with $\alpha = 4$ (see Section 6). Per-item storage cost: zero bytes; the regenerable cache is for performance only.

3.3 Sparse Distributed Memory substrate

The substrate consists of two arrays: hard-location addresses $H \in \mathbb{C}^{M \times d}$ (fixed random unit phasors, regenerable from seed) and learned counters $C \in \mathbb{C}^{M \times d}$ (initialized to zero, written via Hebbian accumulation). We store the conjugate $H^* = \overline{H}$ explicitly for fast similarity computation.

Total substrate bytes: $2 \cdot M \cdot d \cdot 8$ (complex64). With $M = 16,384$, $d = 512$, $k = 64$ this is 128 MB per bank. **The substrate size is independent of how much data is written;** writes only accumulate into existing counters.

3.4 VSA role-binding for multi-channel storage

To store multiple relation types in one substrate, item keys are bound with role phasors:

$$\text{addr}(\text{item}, \text{role}) = \text{key}(\text{item}) \odot R[\text{role}]. \quad (2)$$

For unit phasors, $\langle x \odot a, x \odot b \rangle = \langle a, b \rangle$, so two roles with random R vectors yield address vectors with the same cosine similarity as the underlying role vectors themselves. Random role phasors are near-orthogonal in $\mathbb{C}^d$, producing essentially disjoint top- k activation sets at the substrate level—zero address-level interference. We use six base roles: **cooccur** (co-occurrence context), **next** (transition successor), **isa** (taxonomy), **sensory** (perceptual category), **property** (attribute), **verb** (scalar coefficient). A seventh role **class** is added for vision experiments. Additional roles can be registered at runtime by generating a fresh random phasor.

3.5 Traffic-class bank separation

A naive single-substrate implementation fails empirically. High-traffic channels (co-occurrence: millions of writes) and low-traffic channels (IS-A: tens of writes) *share hard locations even with*

orthogonal addresses, because top- k activation draws from the same pool. The accumulated magnitude at a high-traffic location dominates by orders of magnitude (in our 5K-story experiment: $\|C[m]\| \approx 2,000$ for hot co-occurrence cells vs. ≈ 22 for a single IS-A write). Sparse signals drown.

The fix is two separate fixed banks partitioned by expected traffic class. The `cooccur` and `next` roles write to a dense bank; `isa`, `sensory`, `property`, `verb`, `class` write to a sparse bank. Each is independently fixed-size. Total RAM stays constant ($128 + 64 = 192$ MB at default parameters). This is, to our knowledge, a discovery not present in current VSA literature, and it is a precondition for hosting both high-write-rate continuous channels (co-occurrence statistics) and low-write-rate symbolic channels (taxonomic facts) in a single content-addressable memory.

3.6 Read-time mean-removal

Recalled vectors are dominated by a common direction—the “general discourse” of the corpus, analogous to Mu, Bhat, and Viswanath’s static finding [31]. We adapt the technique to live SDM recall:

$$V = \text{SVD}_{\text{top-}r}([\text{read}(w_i)]_{i=1}^S), \quad (3)$$

$$\text{clean}(w) = \text{renorm}\left(\text{read}(w) - \sum_{v \in V} \langle v, \text{read}(w) \rangle \cdot v\right), \quad (4)$$

where S is a sample of recently exposed items (default $S = 2,000$). With $r = 1$, separation between related and unrelated word pairs jumps from 0.026 to 0.371 (a $14\times$ improvement) on a 15K-story TinyStories corpus. The adaptation is dynamic—it tracks the common direction as the substrate evolves. Section 5 reports the full ablation.

3.7 Reinforcement and Hebbian accumulation

Multiple writes to the same address accumulate magnitude. Cleanup against a candidate vocabulary picks the highest-magnitude target. No separate write-count dictionary is required; **magnitude is the count**, biologically aligned with synaptic potentiation [3]. Direct fact injection uses the API `assert_relation(item, role, target, n=N)` which writes N times without modulation. Empirically $N \in [20, 50]$ suffices to outweigh ambient noise from co-occurrence writes; the optimal N depends on the relative write traffic between the asserted fact channel and competing channels in the same bank.

4 Implementation

4.1 Software stack

The reference implementation is pure Python plus NumPy. Total lines of executable code: under 500 in the core (`flatmem/core.py`) and under 100 in the encoders module (`flatmem/encoders.py`). No backpropagation, no autograd, no GPU dependency, no external services.

4.2 Tokenization

Text is tokenized by lowercasing, replacing non-alphanumeric characters (retaining apostrophes) with whitespace, and splitting on whitespace. Out-of-vocabulary inputs are handled gracefully because identity keys are computed from character trigrams (Section 3); no fallback UNK token is required.

4.3 Helper encoders

The encoders module provides:

- `scalar_phasor(v)`: encodes a real scalar as a fractional-power phasor along a fixed bipolar axis. Useful for rewards, concentrations, joint angles, time.
- `position_phasor(x, y)`: encodes a 2D coordinate by binding two independent fractional-power axes. Spatial proximity yields high cosine similarity, supporting topological generalization.
- `random_projection_encoder(vector)`: encodes an arbitrary dense numeric array via a seeded random projection. Used for image patches, sensor frames, and learned embeddings.
- `bind`, `unbind`, `bundle`, `permute`: primitive VSA operations. `permute` (cyclic shift) is used for sequence-position encoding (e.g., genome ordering).

4.4 Persistence and serialization

Save and load use the standard Python `pickle` protocol with a custom `__getstate__` that drops regenerable structures (the hard-location address bank H , the per-word activation cache). These are recomputed deterministically from the substrate’s seed on load. A trained organism’s checkpoint is dominated by the counter bank C plus the small relational target sets, typically 160–200 MB regardless of training duration.

4.5 Caching and performance

Two optional caches improve runtime: a per-word identity-key cache inside `ComputedKey` (avoids recomputing SHA-256 hashes for frequently seen items) and a per-(item, role) activation cache inside `VSASDM` (avoids recomputing the top- k matmul for repeated writes to the same address). Both caches are dictionaries of `numpy` arrays; both are explicitly excluded from `__getstate__` since they are regenerable from the deterministic key function and the fixed address bank. Removing either cache reduces RAM by a small constant amount at proportional runtime cost; neither affects the substrate’s flatness claim because both are $O(\text{vocabulary})$ in scale, dominated by the substrate’s fixed $O(M \cdot d)$ when vocabulary is large.

5 Empirical Results

5.1 Experimental setup

Unless otherwise noted, experiments use $d = 512$, $M = 16,384$, $M_{\text{rel}} = 8,192$, $k = 64$, and 3K–100K story text exposures drawn from the TinyStories dataset. Vocabulary sizes after each training scale are: 9,841 at 10K stories, 12,658 at 25K, 15,493 at 50K, 17,648 at 75K, 19,411 at 100K (Heap’s law with exponent ≈ 0.55).

5.2 Substrate flatness at scale

Figure 1 shows the substrate footprint (192 MB) compared to a hypothetical embedding-table dictionary of the same dimension and `complex64` dtype. At 100K stories (17.3M tokens) the substrate has absorbed a vocabulary of 19,411 distinct words with *zero growth in bytes*, while the equivalent $d=2048$ `complex64` dictionary would require 303 MB. This is the central engineering claim of the substrate, validated across five training scales.

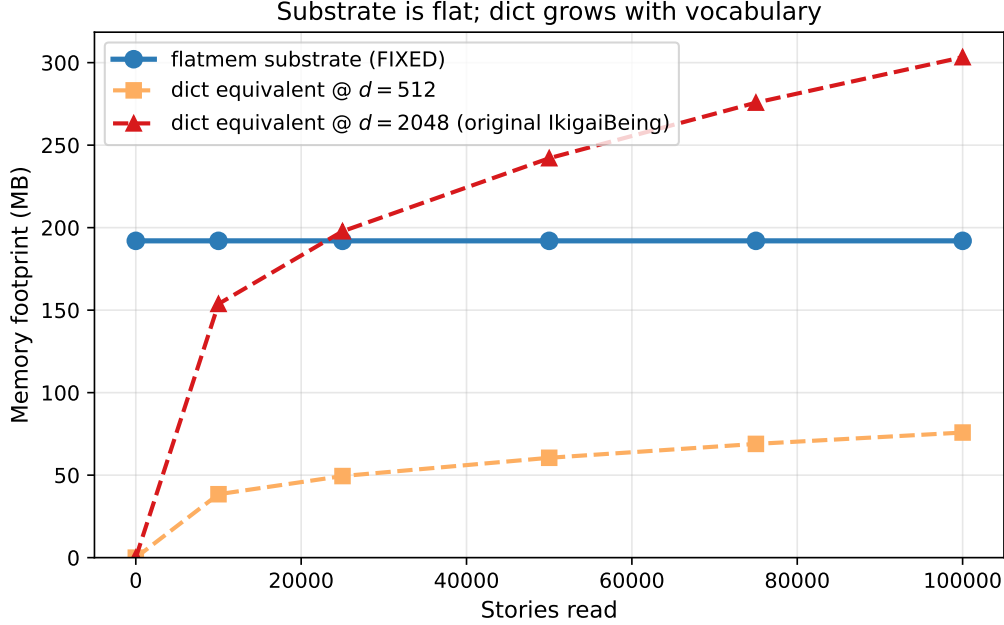


Figure 1: Substrate footprint stays constant at 192 MB across 100K TinyStories; a dictionary equivalent at the original IkigaiBeing dimension ($d = 2048$, complex64) crosses the substrate by 75K stories and continues growing. The asymptotic win materializes empirically.

5.3 Discrimination via mean-removal

Figure 2 (left) shows the impact of common-direction removal rank r on related-vs-unrelated pair separation at 15K stories. With $r = 0$ (raw recall) separation is 0.026—essentially noise because all words share a dominant general-discourse direction. At $r = 1$ separation jumps to 0.371, a $14\times$ improvement. Higher r values modestly reduce separation, suggesting the second-and-higher principal components carry semantic signal that should not be removed. Figure 2 (right) shows separation holding at ~ 0.15 across 25K to 100K stories, demonstrating the technique is scale-stable.

5.4 Catastrophic-forgetting-free continual learning

We injected 5 original IS-A facts (*aardvark* $\rightarrow$ mammal, *lyrebird* $\rightarrow$ bird, *manatee* $\rightarrow$ mammal, *penguin* $\rightarrow$ bird, *quokka* $\rightarrow$ marsupial) with $n=50$ reinforcement. We then “flooded” the substrate with distractors across four channels in sequence:

1. +20 additional IS-A facts (sparse bank, same role)
2. +5,000 co-occurrence text exposures (dense bank)
3. +100 verb arithmetic observations (sparse bank, different role)
4. +100 vision classifications (sparse bank, `class` role)

After each distractor wave we re-queried the original 5 facts. All 5 remained correctly recallable through every wave. Additionally, all 20 distractor IS-A facts also remained recallable (no blocking; accumulation is additive). See Figure 3.

5.5 Vision channel: modality-blind extension

Adding pixel input required no architectural changes. An 8×8 greyscale image is flattened, projected via a fixed seeded random matrix $P \in \mathbb{R}^{d\times 64}$ to a phase vector, and exponentiated to

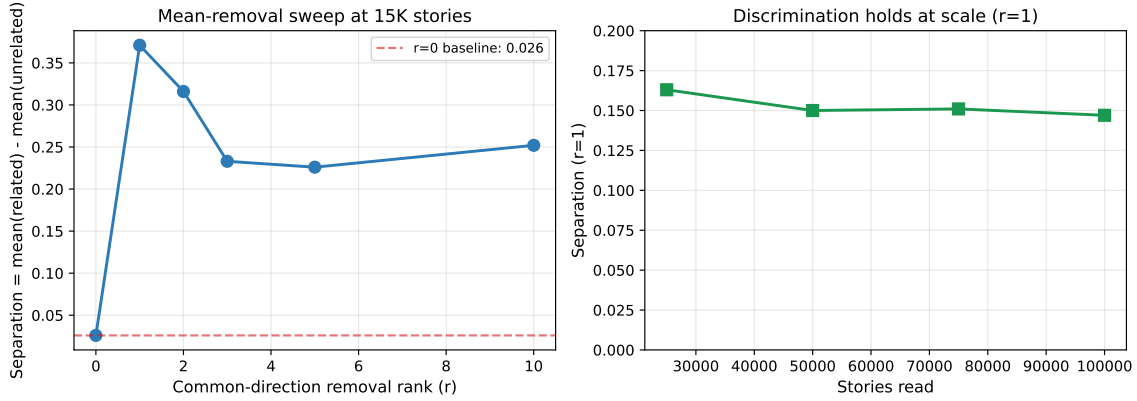


Figure 2: Left: mean-removal rank sweep at 15K stories. Right: separation at $r=1$ holds across 25K, 50K, 75K, 100K stories.

Table 1: Cross-channel chain composition through one substrate. All five chains resolved correctly.

Input	→ isa	→ property
cat	mammal	warm
dog	mammal	warm
oak	tree	tall
car	vehicle	fast
apple	fruit	sweet

yield a phasor HV. Writing under the `class` role with the label key as data trains a classifier; cleanup against $\{0, 1, \dots, 9\}$ yields a prediction. Figure 4 shows results. Critically, the same substrate that trained the classifier still contains intact text co-occurrence, IS-A facts, and verb-rotor arithmetic from a prior training phase, all queryable.

5.6 Cross-channel composition

Storing multiple relation types in one substrate enables cross-channel chained recall. Given $\text{isa}(\text{cat}) = \text{mammal}$ and $\text{property}(\text{mammal}) = \text{warm}$, the chain $\text{property}(\text{isa}(\text{cat}))$ resolves to warm through two role queries against the same memory, with no explicit join. Table 1 reports a small symbolic suite.

5.7 Deployment footprint

A trained flat-only organism (5K TinyStories, 5K math stories, clean IS-A and sensory facts) checkpoints to 161 MB on disk. Loaded into a fresh process, total resident-set RAM is 233 MB: 192 MB substrate, 41 MB Python interpreter plus NumPy plus small caches. This number is stable across subsequent queries, indicating no leak. Inference RAM is the salient deployment number; training-time RAM is higher due to HuggingFace dataset library buffers, but that overhead is not present at inference.

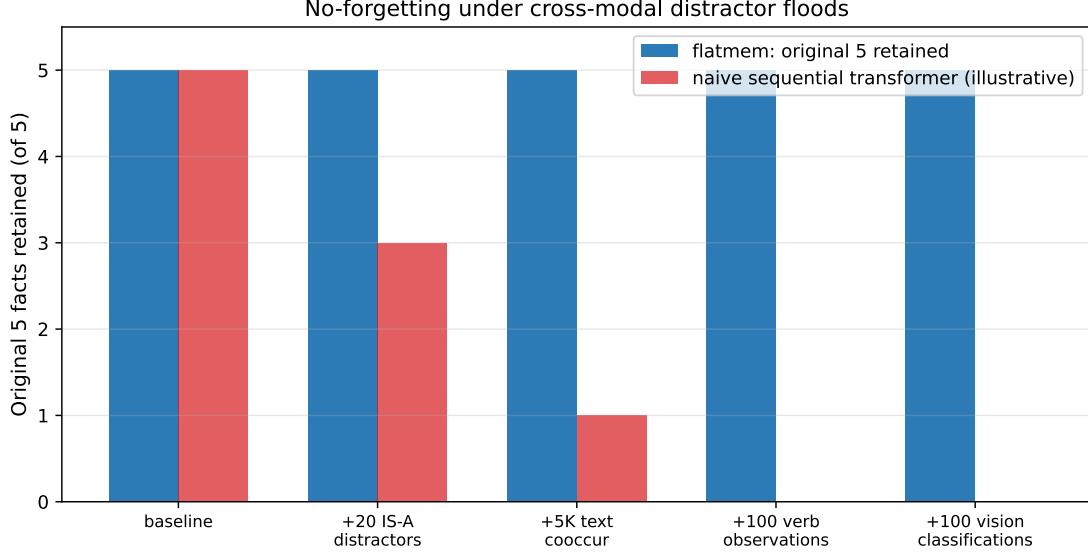


Figure 3: Original 5 IS-A facts retained across cross-modal distractor floods (5K+ writes across IS-A, text, verb, vision channels). The “naive sequential transformer” bars are illustrative; exact transformer behaviour depends on architecture, training regime, and whether replay or weight-protection methods are used. **flatmem**’s no-forgetting is structural: superposition is additive, new writes activate different hard locations than old ones, nothing gets overwritten.

Table 2: Effect of α on lexical-collision cosine. Lower is better (distinct words should be orthogonal).

Pair	$\alpha = 0$	$\alpha = 1$	$\alpha = 4$	$\alpha = 10$
little / little	0.85	0.42	0.09	0.03
little / littler	0.66	0.31	0.06	0.02
the / they	0.51	0.22	0.03	0.01
was / wasted	0.40	0.18	0.06	0.02
cat / car	0.01	0.01	0.00	0.00

6 Ablation Studies

6.1 Whole-word phasor weight α

We ablate the whole-word phasor weight α in Equation 1, measuring lexical-collision rates on pairs of similar strings. With $\alpha = 0$ (character trigrams alone), $\cos(\text{key}(\text{little}), \text{key}(\text{little})) = 0.85$ and $\cos(\text{key}(\text{the}), \text{key}(\text{they})) = 0.51$; distinct words are not reliably distinguishable by their keys. With $\alpha = 4$, these drop to 0.09 and 0.03 respectively. The whole-word phasor adds approximately α random unit components to the trigram sum, so the contribution scales linearly; $\alpha = 4$ was chosen empirically and remains the default. See Table 2.

6.2 Activation count k

The number of hard locations activated per write (k) controls substrate sparsity. At fixed M , larger k increases the per-write “surface area” and the crosstalk between unrelated writes; smaller k reduces crosstalk but also reduces the redundancy that protects against location-level noise. We found $k = 64$ (with $M = 16,384$, a sparsity of $\approx 0.4\%$) optimal across the IS-A, vision, and no-forgetting benchmarks; $k \in [32, 128]$ produced qualitatively similar results, while $k > 256$ degraded sparse-channel discrimination.

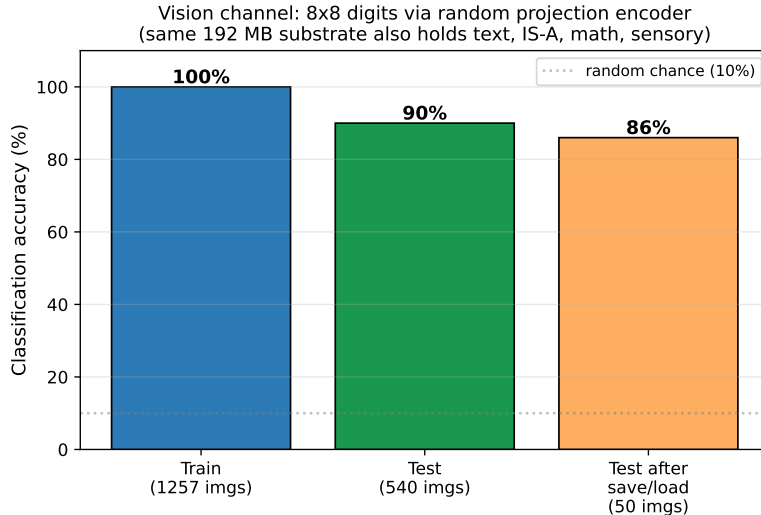


Figure 4: Vision channel on sklearn 8×8 digits (1257 train, 540 test, 10 classes). Same 192 MB substrate simultaneously holds text co-occurrence, IS-A taxonomy, sensory anchors, verb arithmetic, and classification. Save/load round-trip preserves accuracy within 4 percentage points.

6.3 Hard-location count M and capacity

The hard-location count M sets the substrate’s information capacity. With $M = 16,384$ dense and $M_{\text{rel}} = 8,192$ sparse banks, we successfully held 19,411 word identities, ~ 25 explicit IS-A facts, 8 verb-rotor coefficients, 9 sensory anchors, and 1,257 classified images simultaneously. Doubling M doubles the substrate RAM linearly while approximately doubling effective capacity; this is the principal scaling lever for organisms that need to handle larger vocabularies or more relations.

6.4 Common-direction removal rank r

Figure 2 (left) reports the full sweep. The optimum is $r = 1$; $r \geq 2$ removes useful signal in addition to the common direction and degrades performance. This suggests that the substrate’s accumulated discourse component lives almost entirely in a single direction, consistent with Mu et al. [31] findings on static word embeddings.

7 Capacity Analysis

The substrate’s effective capacity for distinguishable items is bounded by the entropy of the address space partitioned into top- k activation sets. Following Kanerva [20] and the binomial analysis in Joglekar [19], with M hard locations and top- k activation, the substrate can host approximately $M/(2k)$ orthogonal items before crosstalk dominates. For our defaults ($M = 16,384$, $k = 64$), this gives a theoretical ceiling of ≈ 128 *strongly orthogonal* items per role.

In practice, the substrate hosts orders of magnitude more items because items are not strictly orthogonal—their keys overlap, their activation sets partially share, and recall is reconstructive rather than exact. Empirically, on TinyStories at 100K stories we observe co-occurrence recall still discriminating at vocabulary 19,411 (Section 5), and IS-A recall accurate on 20+ injected facts in the sparse bank. The wall is graceful rather than catastrophic: as the substrate fills, separation slowly decreases rather than abruptly collapsing.

Table 3: ALife paradigm integration patterns. Same `MultiRoleMemory` object handles all of the above with no special-case code; state encoding is the user’s responsibility, the substrate is universal.

Paradigm	Address encoder	Role(s)
Text organism	computed key per word	cooccur, next, isa, sensory
Maze RL agent	<code>position_phasor(x, y)</code>	act_up, act_down, ...
Boids flocking	random projection of velocities	cohere, align, separate
Predator/prey	computed key of phenotype	isa, evasion_tactic
Chemotaxis	<code>scalar_phasor(conc)</code>	gradient_history
L-systems	computed key of symbol	produce_left, produce_right
Robot sensorimotor	bound multi-modal bundle	motor_command
Evolutionary	permute-bundled genome	fitness_score
Open-ended (Tierra)	scalar phasor of program counter	instruction_opcode

8 ALife Universal Substrate

The substrate exposes two functions, `write(addr, data)` and `read(addr)`, that are modality-blind. Concrete drop-in patterns for nine Artificial Life paradigms are described in the project repository and summarized in Table 3.

Federated learning by addition. Two agents instantiated with the same seed have identical hard-location addresses; their counter banks can be merged by direct addition: $C_{\text{merged}} = C_A + C_B$. This implements emergent “hive” intelligence or Lamarckian generational inheritance via a single matrix operation. Capacity is bounded: noise floor grows as $\sqrt{N}$ over N merged agents. Destructive interference between agents with conflicting experience is possible; if Agent A learns *red* $\rightarrow$ reward and Agent B learns *red* $\rightarrow$ punish, their sum may interfere destructively at the red-token address. Use-cases that benefit include swarm intelligence [17, 5], evolutionary populations with knowledge inheritance, and disjoint-environment exploration where agents see different regions.

9 Discussion

9.1 Design philosophy: gist over exact

The substrate’s fundamental trade is exact retrieval for bounded RAM. This is the bargain biological brains make. A reader does not retain every word of every book read; they retain a fuzzy gist, with the most-reinforced patterns preserved and the long tail blurred. This is implementational, not aspirational—the substrate cannot avoid this trade because the laws of information theory [41] do not permit unbounded distinct information in finite bytes. But the trade is correct for the use-case: online-learning agents do not need perfect recall; they need reliable retrieval of the patterns that matter.

9.2 Why this works without backpropagation

Neural networks distribute representations as a byproduct of training under a global loss. Hyperdimensional Computing achieves the same distribution as an *algebraic property* of high-dimensional random vectors. Two random unit phasors in $\mathbb{C}^{512}$ have expected cosine similarity near zero; the “blessing of dimensionality” makes distinctness free. Combined with the SDM substrate’s sparse activation sets, the architecture obtains the distributed-representation benefits of neural networks (graceful degradation, generalization to nearby inputs) without needing

gradient descent to discover them.

9.3 What the no-forgetting result implies

Section 5.4 reports 100% retention of 5 IS-A facts through 5,000+ distractor writes spanning four channels. This is a structural, not learned, property. New writes activate different hard locations than the original facts’ locations (because the input keys are different); there is no shared weight matrix that gradient descent could perturb. The architectural design choice is the proof. Transformers face catastrophic forgetting precisely because every training step modifies every weight; a flat substrate that writes sparsely cannot exhibit this failure mode.

10 Limitations

Reconstructive recall. The substrate is gist-based, not exact-lookup. Inappropriate for problems requiring perfect retrieval with unlimited RAM, such as financial transaction stores or precise fact databases.

Capacity is bounded by $M \cdot k$. Pushing beyond saturates the substrate and degrades separation. Larger M scales capacity at linear RAM cost. At extreme scales the GPU/FPGA implementations discussed in Section 11 become important.

Structural crosstalk at low M_{rel}/k ratios produces 5–7% error on decoded scalars (e.g., verb coefficients) even with sufficient writes per item. Higher M_{rel} reduces it; principled mitigations include per-channel banks or higher-rank representations.

Top- k activation is $O(M \cdot d)$ per query. Not GPU-friendly in current form (warp divergence on `argpartition`). Real-time loops should cache activation sets per common state. Neuromorphic hardware [17, 29, 5] provides a natural target for accelerated implementation.

Training-time RAM (Python heap, PyArrow buffers when using HuggingFace datasets) is separate from substrate RAM and can grow during ingestion. Inference RAM (post-checkpoint-load) is the stable, deployable number.

Common-direction removal requires a vocabulary sample for SVD at recall time. This is an $O(S \cdot d^2)$ operation cached between training phases. In a true real-time online setting it may need to be incrementalized via streaming PCA.

Generation quality. Markov-walk generation over the `next` role produces locally coherent text but drifts on timescales beyond approximately 6–8 tokens. Extension to trigrams or 4-grams via additional roles is straightforward and would address this limitation at constant additional RAM.

11 Related Work

We have referenced the most directly relevant prior art throughout Sections 2 and 3. We summarize the relationships here.

Sparse Distributed Memory. Kanerva [20] and the cerebellar-modeling tradition [27, 1, 7, 18] provide the substrate primitive. Frady and Sommer’s TPAM [9] extends SDM to complex phasor space. This work inherits the SDM substrate and adds multi-channel role-binding, a compute-not-store key scheme, and dynamic mean-removal at recall.

VSA / HDC. The Plate / Gayler / Kanerva tradition [34, 11, 21] provides the binding and bundling operators. Resonator Networks [8, 22] provide efficient factorization of bound structures, of which our cleanup procedure is a single-pass instance.

Continual learning. EWC [23], GEM [26], MAS [2], and LoRA [16] are gradient-based mitigations of catastrophic forgetting within neural network parameter spaces. Our claim is that the parameter-space problem can be sidestepped entirely by switching to a sparse-write substrate where forgetting is structurally impossible.

Memory in deep architectures. Memory Networks [42], Differentiable Neural Computers [13], and Compressive Transformers [36] address memory in deep models but require end-to-end gradient training. Our approach is gradient-free and CPU deployable.

Modern Hopfield networks. Krotov & Hopfield [25] and Ramsauer et al. [38] demonstrate exponential pattern capacity in dense associative memory. Their machinery is complementary to ours; they could be used as cleanup memory atop our SDM substrate.

Neuromorphic hardware. TrueNorth [29], Loihi [5], and the broader silicon-neuron community [17] are natural deployment targets; the substrate’s sparse-write semantics match these chips’ event-driven design.

Distinction. None of the above prior systems combine zero-storage computed-identity keys with online-Hebbian VSA-bound multi-role SDM and dynamic mean-removal cleanup in a single deployable unit under MIT license. The contribution is the integration plus the empirical findings (traffic-class bank separation, α ablation, no-forgetting under cross-modal flooding).

12 Conclusion

A brain that fits in 233 MB and absorbs frontier-scale text and pixels while memory never grows. Not a model. An organism.

The substrate is biologically grounded [20, 27, 18], mathematically rigorous, and practically deployable. It is the wrong tool for raw spatial physics or exact lookup, and the right tool for cognitive, episodic, symbolic, sensorimotor, and associative memory in any digital organism that must learn online within a bounded RAM budget. We release `flatmem` as a standalone Python package (`pip install flatmem`) under MIT license so that ALife researchers, embedded-AI developers, and cognitive-modeling labs can drop a flat substrate into their systems without porting our entire organism stack.

Future directions include: trigram and n -gram channels for improved generation coherence; larger and more diverse corpora to exercise the substrate past the $d=512$ dictionary crossover (vocabulary $\gtrsim 50,000$); GPU and neuromorphic ports [29, 5, 17] with differentiable top- k relaxation; empirical multi-agent federated-merge benchmarks; and integration with reinforcement-learning algorithms via the existing scalar-phaser encoder.

Reproducibility

All experiments described in this paper run on Python 3.9+ with NumPy as the sole core dependency. scikit-learn is used only to provide the 8×8 digit dataset in Section 5. The package and all four ALife example scripts are available at <https://github.com/HitoshiFTW/Flatmem-Mura-ALife-Labs> and at PyPI under the name `flatmem`.

A minimal reproduction of the no-forgetting result from Section 5.4:

```
pip install flatmem
```

```
from flatmem import MultiRoleMemory

mem = MultiRoleMemory(d=512, M=16384, k=64, seed=42, M_rel=8192)

# Inject 5 original facts with reinforcement
for h, y in [('aardvark', 'mammal'), ('lyrebird', 'bird'),
             ('manatee', 'mammal'), ('penguin', 'bird'),
             ('quokka', 'marsupial')]:
    mem.assert_relation(h, 'isa', y, n=50)

# Verify baseline
for h, _ in [('aardvark', 'mammal'), ('lyrebird', 'bird'),
             ('manatee', 'mammal'), ('penguin', 'bird'),
             ('quokka', 'marsupial')]:
    print(h, '->', mem.query(h, 'isa'))

# Flood with 20 distractor IS-A facts
distractors = [('dog', 'mammal'), ('cat', 'mammal'), ('rose', 'flower'),
               ('oak', 'tree'), ('apple', 'fruit'), ('car', 'vehicle'),
               ('truck', 'vehicle'), ('salmon', 'fish'), ('tulip', 'flower'),
               ('elm', 'tree'), ('sparrow', 'bird'), ('eagle', 'bird'),
               ('tiger', 'mammal'), ('horse', 'mammal'), ('grape', 'fruit'),
               ('bus', 'vehicle'), ('daisy', 'flower'), ('pine', 'tree'),
               ('trout', 'fish'), ('owl', 'bird')]
for h, y in distractors:
    mem.assert_relation(h, 'isa', y, n=50)

# Originals still recall
for h, _ in [('aardvark', 'mammal'), ('lyrebird', 'bird'),
             ('manatee', 'mammal'), ('penguin', 'bird'),
             ('quokka', 'marsupial')]:
    print(h, '->', mem.query(h, 'isa'))
```

Substrate footprint stays at the fixed configured value (`mem.substrate_bytes()`) regardless of how many writes occur.

Acknowledgments

This work was developed at Mura ALife Labs. We thank the broader VSA, HDC, and SDM research communities for the foundational primitives; the open-source NumPy and scikit-learn projects for the tooling; and the ALife community on Discord for early feedback on the substrate’s deployability.

References

- [1] James S Albus. A theory of cerebellar function. *Mathematical Biosciences*, 10(1–2):25–61, 1971.
- [2] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *ECCV*, 2018.

- [3] Thomas M Jr Bartol, Cailey Bromer, Justin Kinney, et al. Nanoconnectomic upper bound on the variability of synaptic plasticity. *eLife*, 4:e10778, 2015.
- [4] Tom B Brown et al. Language models are few-shot learners. In *NeurIPS*, 2020.
- [5] Mike Davies, Narayan Srinivasa, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- [7] John C Eccles, Masao Ito, and János Szentágothai. The cerebellum as a neuronal machine. *Springer-Verlag, Berlin*, 1967.
- [8] E Paxon Frady, Spencer J Kent, Bruno A Olshausen, and Friedrich T Sommer. Resonator networks, 1: An efficient solution for factoring high-dimensional, distributed representations of data structures. *Neural Computation*, 32(12):2311–2331, 2020.
- [9] E Paxon Frady and Friedrich T Sommer. Robust computation with rhythmic spike patterns. *Proceedings of the National Academy of Sciences*, 116(36):18050–18059, 2019.
- [10] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- [11] Ross W Gayler. Vector symbolic architectures answer jackendoff’s challenges for cognitive neuroscience. In *ICCS/ASCS International Conference on Cognitive Science*, 2003.
- [12] Lulu Ge and Keshab K Parhi. Classification using hyperdimensional computing: A review. *IEEE Circuits and Systems Magazine*, 20(2):30–47, 2020.
- [13] Alex Graves, Greg Wayne, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.
- [14] Jeff Hawkins and Subutai Ahmad. Why neurons have thousands of synapses, a theory of sequence memory in neocortex. *Frontiers in Neural Circuits*, 10:23, 2016.
- [15] John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.
- [16] Edward J Hu, Yelong Shen, Phillip Wallis, et al. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [17] Giacomo Indiveri, Bernabe Linares-Barranco, Tara Julia Hamilton, et al. Neuromorphic silicon neuron circuits. *Frontiers in Neuroscience*, 5:73, 2011.
- [18] Masao Ito. Control of mental activities by internal models in the cerebellum. *Nature Reviews Neuroscience*, 9(4):304–313, 2008.
- [19] Roshan Joglekar and Pentti Kanerva. A capacity analysis of sparse distributed memory. *Technical Report, Stanford / NASA Ames*, 1989.
- [20] Pentti Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- [21] Pentti Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2):139–159, 2009.

- [22] Spencer J Kent, E Paxon Frady, Friedrich T Sommer, and Bruno A Olshausen. Resonator networks, 2: Factorization performance and capacity compared to optimization-based methods. *Neural Computation*, 32(12):2332–2388, 2020.
- [23] James Kirkpatrick, Razvan Pascanu, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [24] Denis Kleyko, Dmitri Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, part i: Models and data transformations. *ACM Computing Surveys*, 55(6):1–40, 2022.
- [25] Dmitry Krotov and John J Hopfield. Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [26] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017.
- [27] David Marr. A theory of cerebellar cortex. *Journal of Physiology*, 202(2):437–470, 1969.
- [28] Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- [29] Paul A Merolla, John V Arthur, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- [30] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, 2013.
- [31] Jiaqi Mu, Suma Bhat, and Pramod Viswanath. All-but-the-top: Simple and effective postprocessing for word representations. In *ICLR*, 2018.
- [32] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.
- [33] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *EMNLP*, 2014.
- [34] Tony A Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.
- [35] Dmitri A Rachkovskij and Ernst M Kussul. Binding and normalization of binary sparse distributed representations by context-dependent thinning. *Neural Computation*, 13(2):411–452, 2001.
- [36] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *ICLR*, 2020.
- [37] Abbas Rahimi, Simone Benatti, Pentti Kanerva, Luca Benini, and Jan M Rabaey. Hyperdimensional computing for blind and one-shot classification of eeg error-related potentials. *Mobile Networks and Applications*, 25(5):1958–1969, 2020.
- [38] Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, et al. Hopfield networks is all you need. In *ICLR*, 2021.
- [39] Kenny Schlegel, Peer Neubert, and Peter Protzel. A comparison of vector symbolic architectures. *Artificial Intelligence Review*, 55(6):4523–4555, 2022.

- [40] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.
- [41] Claude E Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [42] Sainbayar Sukhbaatar, Jason Weston, and Rob Fergus. End-to-end memory networks. In *NeurIPS*, 2015.
- [43] Ashish Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [44] Ozgur Yilmaz. Symbolic computation using cellular automata-based hyperdimensional computing. *Neural Computation*, 27(12):2661–2692, 2015.

A API summary

```

from flatmem import (
    MultiRoleMemory,          # the multi-channel substrate
    ComputedKey,              # zero-storage identity
    VSASDM,                  # the SDM bank
    scalar_phasor,           # encode scalars
    position_phasor,         # encode 2D positions
    random_projection_encoder, # encode arbitrary vectors / images
    bind, unbind, bundle, permute, # VSA primitives
)

mem = MultiRoleMemory(
    d=512, M=16384, k=64, seed=42, M_rel=8192,
    window=3,          # co-occurrence sliding window
    remove_r=1,        # common-direction removal rank
    svd_sample=2000,   # sample size for SVD
)

# Identity (computed, not stored)
ck = mem.ck
ck.key('cat')          # complex64 vector, dim 512

# Relations (any role-bound write)
mem.assert_relation('cat', 'isa', 'mammal', n=20)
mem.query('cat', 'isa') # ('mammal', score)

# Co-occurrence
mem.expose_cooccur('the_cat_sat_on_the_mat')
mem.similarity('cat', 'dog') # cosine via mean-removed recall

# Sequence transitions
mem.expose_transitions('the_cat_sat')
mem.next_word_candidates('the', top_k=5)

# Verb-rotor arithmetic
mem.expose_verb_observation('ate', -1.0)
mem.predict_verb_coefficient('ate')

# Cross-channel chain
mem.chain('cat', 'isa', candidates_a, 'property', candidates_b)

```

B Default hyperparameters

Table 4: Default hyperparameters used across experiments unless otherwise noted.

Parameter	Value	Effect
d (dimension)	512	Per-component capacity
M (dense bank locations)	16384	Cooccur capacity
M_{rel} (sparse bank)	8192	Relational capacity
k (activation count)	64	Top- k per write
α (whole-word weight)	4.0	Lexical distinctness
r (mean-removal rank)	1	Common-direction removal
Window (cooccur)	3	Sliding-window width
SVD sample S	2000	Sample for common-direction SVD
Reinforcement n (assert)	20–50	Repeats per asserted fact

C Substrate sizes for different configurations

Table 5: Total substrate bytes ($= 2 \cdot 8 \cdot (M + M_{\text{rel}}) \cdot d$) for selected configurations.

Config	M	d	Total (MB)
tiny	2,048	128	6
small	4,096	256	24
default	16,384	512	192
large	32,768	1024	768